NAME = MOHIT MALVIYA

ROLL NO. = 23

DIVISION = 4CE_A

SUBJECT = PROGRAMMING IN PYTHON

ASSIGNMENT = 3

Github Link

Numerical Computing in Python

🔢 **NumPy** (Numerical Python) – *Numerical Computing*

- **What it is**: A foundational library for numerical operations in Python.
- **Why it's useful**: It allows fast computation on large arrays and matrices.
- **Key features**:
  - Supports multidimensional arrays.
  - Provides mathematical functions like linear algebra, Fourier transforms, statistics.
- **Example**:

python Copy Edit

```python
import numpy as np
arr = np.array([1, 2, 3])
print(np.mean(arr))  # Output: 2.0
```

Pandas – Data Manipulation in Python

📊 Pandas – *Data Manipulation*

- **What it is**: A library for handling and analyzing data structures, especially tabular data.
- **Why it's useful**: Great for working with data in CSV, Excel, SQL, etc.
- **Key features**:
  - DataFrame and Series objects.
  - Powerful group-by and pivot operations.
- **Example**:

python Copy Edit

```python
import pandas as pd
df = pd.DataFrame({'Name': ['Alice', 'Bob'], 'Age': [24, 27]})
print(df['Age'].mean())  # Output: 25.5
```

Matplotlib / Seaborn – Data Visualization

📈 **Matplotlib / Seaborn** – *Data Visualization*

- **What they are:**
  - **Matplotlib**: A basic plotting library.
  - **Seaborn**: Built on Matplotlib, adds prettier and more complex visualizations.
- **Why useful**: Makes it easy to create charts and graphs for data analysis.
- **Example (Seaborn)**:

python Copy Edit

```python
import seaborn as sns
import matplotlib.pyplot as plt
sns.set(style="darkgrid")
sns.barplot(x=['A', 'B', 'C'], y=[4, 7, 5])
plt.show()
```

Scikit-learn – Machine Learning

🤖 Scikit-learn – *Machine Learning*

- **What it is:** A powerful library for implementing machine learning algorithms.
- **Why it's useful:** Easy to use for classification, regression, clustering, etc.
- **Key features:**
  - Built-in models: decision trees, SVMs, k-NN, etc.
  - Preprocessing, cross-validation, model evaluation.
- **Example:**

python Copy Edit

```python
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit([[1], [2], [3]], [2, 4, 6])
print(model.predict([[4]]))  # Output: [8.]
```

TensorFlow / PyTorch – Deep Learning

🧠 TensorFlow / PyTorch – *Deep Learning*

- **What they are**:
  - **TensorFlow** (by Google) and **PyTorch** (by Meta) are frameworks for building and training neural networks.
- **Why they're useful**: Used for deep learning tasks like image recognition, NLP, and AI applications.
- **Key features**:
  - GPU support for faster training.
  - Flexibility for building custom models.
- **Example (PyTorch)**:

python Copy Edit

```python
import torch
x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
y = x ** 2
y.backward(torch.ones_like(x))
print(x.grad)  # Output: tensor([2., 4., 6.])
```

OpenCV – Computer Vision

👁️ OpenCV – *Computer Vision*

- **What it is**: A library for real-time image and video processing.
- **Why it's useful**: Can detect faces, objects, and perform image filtering.
- **Key features**:
  - Read, write, and manipulate images/videos.
  - Image transformation, edge detection, face/object tracking.
- **Example**:

python Copy Edit

```python
import cv2
image = cv2.imread('image.jpg')
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
cv2.imshow('Gray Image', gray)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

NLTK / spaCy – Natural Language Processing (NLP)

NLTK / spaCy – *Natural Language Processing (NLP)*

- **What they are**:
  - **NLTK** (Natural Language Toolkit): Ideal for teaching and research.
  - **spaCy**: Industrial-strength NLP library for faster performance.
- **Why they're useful**: Handle tasks like text classification, tokenization, and named entity recognition (NER).
- **Example (spaCy)**:

python Copy Edit

```python
import spacy
nlp = spacy.load("en_core_web_sm")
doc = nlp("Dhruvi is working on a Python project.")
for ent in doc.ents:
    print(ent.text, ent.label_)
```

Pillow – Image Processing

🖼️ **Pillow** – *Image Processing*

- **What it is**: A Python Imaging Library (PIL) fork used for opening, editing, and saving images.
- **Why it's useful**: Ideal for basic image processing tasks like resizing, cropping, filtering, adding text, etc.
- **Key features**:
  - Supports many image formats (JPEG, PNG, BMP, GIF, etc.)
  - Easy to manipulate pixels and apply effects
  - Can generate thumbnails, convert formats, and overlay images
- **Example**:

python Copy Edit

```python
from PIL import Image

# Open an image
img = Image.open("example.jpg")

# Resize the image
img_resized = img.resize((100, 100))

# Save the image
img_resized.save("resized.jpg")
```

Keras – High-Level Deep Learning API

🧠 Keras – *High-Level Deep Learning API*

- **What it is:** A high-level neural networks API, written in Python and capable of running on top of TensorFlow.
- **Why it's useful:** Makes building and training deep learning models easy and beginner-friendly.
- **Key features:**
  - Simple and intuitive syntax.
  - Pre-trained models for image and text tasks.
  - Works well for fast prototyping.
- **Example:**

python Copy Edit

```python
from keras.models import Sequential
from keras.layers import Dense

# Create a simple neural network
model = Sequential([
    Dense(32, activation='relu', input_shape=(10,)),
    Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

TensorFlow – Deep Learning Framework

🧠 TensorFlow – *Deep Learning Framework*

- **What it is:** An open-source library developed by **Google** for machine learning and deep learning.
- **Why it's useful:**
  - Highly flexible and powerful.
  - Supports both low-level operations and high-level APIs (like Keras).
  - Used in real-world AI applications: image recognition, NLP, etc.
- **Fun fact:** TensorFlow supports GPU acceleration, making training much faster.

✅ Short Example: Linear Regression with TensorFlow

python Copy Edit

```python
import tensorflow as tf
import numpy as np

# Sample data
X = np.array([0, 1, 2, 3, 4], dtype=float)
Y = np.array([0, 2, 4, 6, 8], dtype=float)  # y = 2x

# Build the model
model = tf.keras.Sequential([
    tf.keras.layers.Dense(units=1, input_shape=[1])
])

# Compile the model
model.compile(optimizer='sgd', loss='mean_squared_error')

# Train the model
model.fit(X, Y, epochs=100, verbose=0)

# Predict a new value
print(model.predict([5]))  # Expected output: ~10
```

THANK YOU!